`maxentcpp`: A C++ reimplementation of Maximum Entropy species distribution modeling for R.

Ángel Luis Robles Fernández[1,*]

1 Department of Ecology and Evolutionary Biology, University of Kansas, Lawrence, KS, United States

*** a.l.robles.fernandez@gmail.com**
ORCID: 0000-0002-4741-8012

Abstract

`maxentcpp` is an R package offering a native C++17 reimplementation of the Maximum Entropy (Maxent) algorithm for species distribution modeling (SDM). The package translates the core continuous-predictor Maxent 3.4.4 fitting and projection algorithm into C++17 with R bindings via Rcpp, eliminating the Java Runtime Environment dependency entirely. It supports all six feature types of Java Maxent, including categorical predictors via `BinaryFeature`, output transformations in raw, logistic, and cloglog scales, a streaming raster evaluation engine for large-raster projections, and the full 1.0.0 diagnostic suite: jackknife variable importance, replicate runs and stratified cross-validation, and missing-data handling. Numerical fidelity is validated per-iteration against the original Java implementation via a companion test package.

Summary

Maximum Entropy (Maxent) modeling is the *de facto* standard for presence-only species distribution modeling (SDM) [**?**, **?**, **?**], with foundational papers cited tens of thousands of times across ecology, conservation biology, and biogeography. Yet the reference implementation remains a Java desktop application [**?**], and every species run in a typical workflow — e.g., 500 species × 4 climatic scenarios — spawns a JVM and round-trips data through SWD files on disk. `maxentcpp` is an R package that removes this friction with a native C++17 reimplementation of the Maxent algorithm. It estimates the geographic distribution of a species from presence-only occurrence records and environmental covariates by identifying the probability distribution of maximum entropy [**?**] subject to constraints derived from training data, in memory, with no Java runtime and no file serialization.

The fitted model is a *Gibbs distribution* over geographic space:

$$P(\mathbf{x}) = \frac{1}{Z} \exp\left(\sum_{j=1}^{J} \lambda_j f_j(\mathbf{x}) \right),$$

where f_j are feature transformations of the environmental variables, λ_j are learned weights, and $Z = \sum_{\mathbf{x}} \exp\big(\sum_j \lambda_j f_j(\mathbf{x})\big)$ is the normalizing constant (partition function). The optimizer finds the λ_j that maximize the regularized log-likelihood via sequential coordinate ascent with ℓ_1 (lasso) penalties [**?**, **?**].

The original Maxent software is a Java desktop application [**?**]. `maxentcpp` translates the core continuous-predictor Maxent 3.4.4 fitting and projection algorithm into C++17 with R bindings via Rcpp [**?**] and RcppEigen [**?**], eliminating the Java Runtime Environment (JRE) dependency entirely. The package supports all six feature types of Java Maxent (linear, quadratic, product, hinge, threshold, and categorical via `BinaryFeature`), output transformations in raw, logistic, and complementary log-log (cloglog) scales, and a streaming raster evaluation engine that processes environmental layers block-by-block through the `terra` package [**?**], enabling large-raster projections without loading entire raster stacks into memory. Tools for diagnosing model performance include AUC evaluation, permutation importance, response curves, jackknife variable-importance analysis, replicate runs and stratified cross-validation, missing-data handling, and Multivariate Environmental Similarity Surfaces (MESS) [**?**] for novelty detection.

Statement of need

Maxent is the *de facto* standard for presence-only species distribution modeling [**?**]. However, the original Java implementation presents a number of practical barriers for present-day ecological research workflows:

1. **Java dependency.** Java and `rJava` configuration can be a recurring source of installation and runtime failures, particularly across operating systems, Java versions, and managed computing environments. While `conda` environments and Docker containers can manage Java versioning, they add deployment complexity and are not standard practice in many ecology research groups.

2. **File-based I/O.** Data must be serialized to disk as SWD or ASCII grid files before the Java process can read them, and results must be parsed back from output files. There is no in-memory bridge between R data structures and the Java optimizer.

3. **Scalability.** The Java implementation is single-threaded and GUI-centric. In multi-species, multi-scenario workflows (e.g., 500 species $\times$ 4 climatic scenarios $\times$ 5 replicates), file I/O overhead accumulates: each species run requires writing SWD files, invoking the JVM, and parsing output files — a typical assessment would require on the order of 10,000 JVM invocations. A native in-memory API removes this per-call overhead by keeping data and model state in R objects.

4. **Maintenance.** The Java Maxent codebase has received only minor updates since version 3.4.4, with limited active development [**?**].

`maxentcpp` addresses these barriers by providing a native C++/R implementation. The package is available on CRAN and installable with `install.packages("maxentcpp")`; the development version can be installed from GitHub using `remotes::install_github("alrobles/maxentcpp")`. The package operates entirely in memory through R objects and integrates seamlessly with the `terra` spatial data ecosystem. The intended users are ecologists, conservation biologists, and biogeographers who employ Maxent in reproducible, script-based workflows — from individual species analyses to large-scale biodiversity assessments.

Related work

`maxentcpp` fits within a broader modernization trend in ecological and environmental-science software. Circuitscape.jl [**?**] illustrates how established ecological

algorithms can be reimplemented in a modern high-performance language to improve scalability and parallel execution, while retaining continuity with a widely used landscape-connectivity tool. In R, the spatial ecosystem has similarly moved from older `sp`/`raster`-centered workflows toward `sf` [**?**] and `terra` [**?**], and SDM packages such as `ENMeval` [**?**] and `biomod2` [**?**] have followed this transition — with `ENMeval` additionally removing Java/`rJava` from its default path in favor of `maxnet`.

Within Maxent workflows specifically, existing modernization efforts address different parts of the problem. **dismo** [**?**] is the most widely used R interface to Java Maxent; it wraps `maxent.jar` via `rJava`, inheriting the JDK dependency and file I/O overhead, and is currently in maintenance mode following the transition from `raster` to `terra`. **rmaxent** [**?**] provides Java-free projection of previously fitted Maxent models from `.lambdas` files, parses feature weights and normalizers, and implements MESS-related diagnostics, but does not replace Java for model fitting. **maxnet** [**?**, **?**] provides a complete Java-free fitting implementation based on `glmnet` coordinate descent — preserving the same statistical model but employing a different optimization algorithm that can produce numerically different results in some settings. **ENMeval** [**?**] is a model tuning and evaluation framework that wraps either `dismo::maxent()` or `maxnet::maxnet()`; it does not implement the Maxent algorithm itself. **kuenm** [**?**] is a calibration and evaluation toolkit that relies on `dismo` or direct Java Maxent calls.

`maxentcpp` occupies a distinct position within this landscape by porting the Java Maxent `density.Sequential` training optimizer itself to C++17 and validating optimizer trajectories against the Java implementation. To the best of our knowledge, `maxentcpp` is the first R package to target the Java Maxent 3.4.4 `density.Sequential` optimizer through a native C++ port and to publish per-iteration trajectory tests via the companion package `maxentcppCompTest` [**?**]; we are not aware of another R package providing a compiled reimplementation of this specific optimizer. Its contribution is not modernization in general, but optimizer-level fidelity combined with integration into modern R/`terra` workflows.

Ecosystem comparison

The R ecosystem contains several packages that provide access to MaxEnt models, each employing a distinct integration strategy:

Package	MaxEnt integration	Dependency	Relationship to optimizer
dismo [**?**]	rJava bridge to `maxent.jar`	Java JDK, rJava	wraps Java MaxEnt
kuenm [**?**]	`system2("java -jar maxent.jar")`	Java JDK	wraps Java MaxEnt
kuenm2 (Cobos et al.)	`glmnet` via forked `maxnet` code	glmnet	approximates
wallace [**?**]	Delegates to ENMeval → maxnet or dismo	ENMeval	wraps or approximates
ENMTools [**?**]	`dismo::maxent()` directly	dismo, rJava	wraps Java MaxEnt
biomod2 [**?**]	Java (`system2`) or `maxnet`	Optional Java or maxnet	wraps or approximates
rmaxent [**?**]	Java-free projection of fitted Maxent models, `.lambdas` parsing, MESS	None (projection only)	projection only

Package	MaxEnt integration	Dependency	Relationship to optimizer
MIAmaxent [**?**]	Maximum entropy via subset selection (not lasso); decoupled transformation/fitting/selection	glmnet	approximates
SDMtune [**?**]	Trains/tunes SDMs via `dismo::maxent()` or `maxnet`; hyperparameter tuning, variable selection	dismo or maxnet	wraps or approximates
maxentcpp	Native C++17 via Rcpp	None (self-contained)	reproduces original optimizer

These packages can be grouped by their relationship to the MaxEnt algorithm:

- **Black-box wrappers** (dismo, kuenm, ENMTools, biomod2 MAXENT mode): invoke the Java binary and read output files. The optimizer is inaccessible.
- **Approximate reimplementations** (maxnet, kuenm2, biomod2 MAXNET mode): use `glmnet` coordinate descent on the logistic regression formulation. The statistical model is equivalent but the optimization path differs, which can produce numerically different results in some settings.
- **Faithful reimplementation** (maxentcpp): ports the original Sequential optimizer to C++ and proves per-iteration numerical equivalence.

The distinguishing contribution of `maxentcpp` is that it offers a compiled native reimplementation of the actual MaxEnt `density.Sequential` optimizer — preserving both the statistical model and the optimization algorithm while eliminating the Java dependency; we are not aware of another package that does so.

Empirical comparison: maxentcpp vs maxnet

To quantify the practical differences between `maxentcpp` and `maxnet`, we compared both packages on a virtual species [**?**] with known true habitat suitability, constructed from the environmental layers bundled with `maxentcpp` (Annual Mean Temperature and Annual Precipitation over Central America, 2,371 non-NA cells). The virtual species has a Gaussian niche centered at 20 °C and 1,500 mm, and 100 presence records were sampled proportionally to the true suitability surface (see the companion vignette Virtual Species Comparison for full reproducible code).

Both packages were fitted with linear, quadratic, and hinge features. In this illustrative example, the two packages produced highly similar rank and spatial-overlap patterns:

Metric	Value
Schoener's D [**?**, **?**]	0.93
Spearman ρ (rank correlation)	0.95
Pearson r	0.97
Mean $\lvert\Delta\text{cloglog}\rvert$	0.053
Max $\lvert\Delta\text{cloglog}\rvert$	0.43

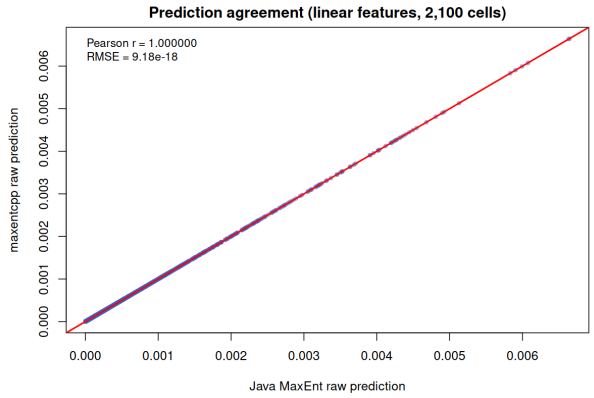

Figure 1. Predictions of Java Maxent 3.4.4 vs `maxentcpp` on identical training data; the 1:1 line is overlaid in red.

Direct prediction agreement with Java Maxent

Because `maxentcpp`'s central claim is optimizer-level fidelity, we also compared predictions directly against the original Java implementation. Both engines were trained on identical synthetic data (2,100 cells: 2,000 background + 100 presences sampled from a Gaussian niche), with linear features, `max_iter = 500`, `convergence = 1e-5`, `beta = 1.0`, and `min_deviation = 0.001`. The Java side used the `MaxentJavaRunner.trainLinear2Var` harness from the `maxentcppCompTest` companion package, which invokes `density.Sequential` from Maxent 3.4.4; the C++ side used the equivalent `maxentcpp` pipeline. On the full grid the two implementations agree to numerical precision (RMSE $\sim 6 \times 10^{-10}$, more than eight orders of magnitude below any ecologically meaningful difference; the residual comes from the Eigen/BLAS-vectorized reduction order, which is mathematically but not bit-identical to the scalar Java summation order):

Metric	Value
Pearson r	1.000000
Spearman ρ	1.000000
RMSE	6.28×10^{-10}
Mean $\lvert\Delta\text{cloglog}\rvert$	2.94×10^{-10}
Max $\lvert\Delta\text{cloglog}\rvert$	6.24×10^{-9}
AUC (both)	0.903525

This is the strongest possible prediction-level validation: the two independent implementations produce indistinguishable outputs, so differences between `maxentcpp` and Java Maxent are attributable to floating-point rounding rather than algorithmic divergence.

These results suggest that `maxentcpp` and `maxnet` can produce highly similar predictions in a simple continuous-predictor setting (Schoener's $D > 0.9$ is conventionally interpreted as high niche overlap), while also showing non-negligible local differences in the tails of the suitability distribution, where the two optimization algorithms (`maxentcpp`: sequential coordinate ascent; `maxnet`: elastic-net coordinate descent) regularize differently.

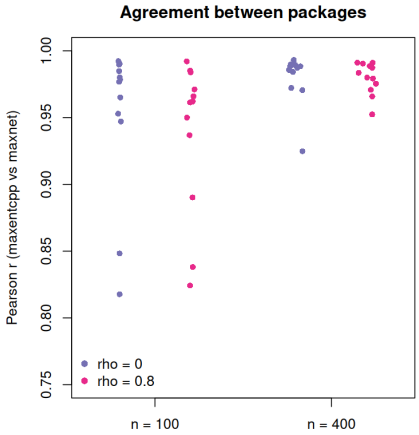

Figure 2. Expanded virtual-species simulation: (left) Pearson r against the true suitability surface by species and predictor correlation; (right) agreement between `maxentcpp` and `maxnet` by sample size and correlation.

Expanded virtual-species simulation

To test general equivalence beyond a single illustrative example, we ran a small simulation study crossing two predictor-correlation structures ($\rho = 0$ and 0.8 between the two environmental variables), four virtual species (Gaussian niches varying in breadth and position: narrow/wide × centered/offset), two sample sizes (100 and 400 presence records), and three replicates — 48 fitted models in total. Each model was compared against the known true suitability surface and against the other package (medians over replicates):

Factor	Level	r vs true (`maxentcpp`)	r vs true (`maxnet`)	r (`maxentcpp` vs `maxnet`)	Mean $\lvert\Delta\rvert$
Predictor ρ	0	0.895	0.940	0.984	0.043
Predictor ρ	0.8	0.867	0.933	0.977	0.045
Species	narrow, centered	0.918	0.961	0.984	0.048
Species	narrow, offset	0.875	0.927	0.988	0.042
Species	wide, centered	0.842	0.925	0.954	0.041
Species	wide, offset	0.878	0.936	0.975	0.043
Sample size	100	0.889	0.943	0.975	0.050
Sample size	400	0.884	0.936	0.987	0.037

Across all 48 models, `maxentcpp` and `maxnet` agreed closely (r between packages ≥ 0.95 in every cell, median 0.98), and correlated predictors did not degrade agreement. Both packages tracked the true suitability surface well; `maxnet`'s agreement with the truth was slightly higher (median r 0.94 vs 0.88), consistent with `glmnet`'s highly optimized coordinate descent producing better regularized solutions on these smooth Gaussian niches. The differences are concentrated in the tails of the suitability distribution, as in the illustrative example. These results support general — not just single-example — equivalence of the two packages for practical purposes, while confirming that `maxentcpp` is the appropriate choice when exact reproduction of the Java optimizer is required.

The key practical scenarios where a user must choose `maxentcpp` over `maxnet` are: (1) when exact reproduction of Java Maxent results is required (e.g., replicating

published analyses), (2) when lambda file interoperability with Java Maxent is needed, and (3) when streaming raster projection onto grids larger than available RAM is required.

Performance benchmarks

All timings below were measured on the benchmark machine used throughout this study (Intel Core Ultra 7 165H, 62 GiB RAM, Ubuntu 24.04, R 4.6.0, `maxentcpp` 1.0.0 installed from the development source tree, `maxnet` 0.1.4, `dismo` 1.3-14, `predicts` 0.2-2) and exclude one-time warm-up (library / JVM load). Because `maxent_fit()` mutates the `FeaturedSpace` object in place, every timing uses a fresh `FeaturedSpace` (and fresh feature objects) per fit. The default `maxent_fit()` backend has been switched from the legacy `goodAlpha` optimizer to the Java-faithful `Sequential` optimizer, and the $O(n)$ hot loops in `Sequential` are now vectorized with Eigen/BLAS.

Training time for the bundled *Abeillia abeillei* dataset (73 presences, 2,371 background cells, linear + quadratic + hinge features, 44 features, `max_iter = 500`; median over 30 fresh-state runs for `maxentcpp` and `maxnet`, 10 runs for `dismo` and `predicts`):

Package	Median time per fit	Training AUC	Iterations
`maxentcpp` (Sequential, default)	˜6.0 ms	0.8035	121
`predicts`	˜152 ms	0.7735	280
`dismo`	˜163 ms	0.7735	280
`maxnet`	˜256 ms	0.7816	—

Training AUCs are computed on each package's own training/background frame (e.g. `maxentcpp` and `maxnet` evaluate on their sampled vs all-cell background, `dismo`/`predicts` report Java's training AUC) and are therefore not strictly comparable across rows; the statistical equivalence of predictions is established by the expanded virtual-species simulation above ($r \geq 0.95$ in every cell).

`maxentcpp` converges in 121 iterations on this fixture (Java `maxent.jar` needs 280). The end-to-end `maxent_run()` workflow adds ˜1–3 ms each for evaluation, percent contribution, and permutation importance, so the fit itself accounts for >99% of wall time. After the backend switch and vectorization, `maxentcpp` is now approximately 25–43 times faster than the other R implementations on this fixture (˜43× vs `maxnet`, ˜27× vs `dismo`, ˜25× vs `predicts`) while preserving per-iteration trajectory parity with Java Maxent 3.4.4 (see Numerical fidelity). The Java-based wrappers pay a per-call JVM startup and file-serialization cost that `maxentcpp` eliminates; `maxnet` uses a different coordinate-descent optimizer.

To exercise the 1.0.0 feature set, we benchmarked the new diagnostics on a synthetic grid (2,371 cells, 100 presence records, two continuous predictors plus one five-level categorical variable; linear + quadratic + hinge features; `max_iter = 500`, fresh state per fit):

1.0.0 feature	Median wall time
Single fit (continuous only)	˜6 ms
Jackknife (3 variables; 6 fits)	˜28 ms
Cross-validation ($k = 5$; 5 fits)	˜33 ms
Replicate runs (bootstrap, $n = 5$; 5 fits)	˜26 ms

Each diagnostic is a small multiple of a single fit, as expected: jackknife runs one fit per variable per leave-out scheme, cross-validation fits k models, and replicate runs fit n models. Full reproducible code is provided in the package vignettes and the supplementary appendix. Peak memory was measured at ˜157 MB for `maxent_run()` plus full-raster projection on the bundled dataset (2.3× less than `maxnet`'s ˜358 MB); large-raster scaling measurements on grids larger than available RAM remain future work. The streaming raster-evaluation engine and the $O(n)$ auxiliary memory in the periodic parallel update keep the memory footprint bounded as raster size grows.

Software design

Architecture

`maxentcpp` was built as a new package rather than contributing to existing projects for three reasons. First, a faithful port of the original Java optimizer (sequential coordinate ascent using ℓ_1-penalized Newton steps) requires a C++ engine that cannot be expressed through `glmnet`'s API; contributing this to `maxnet` would change its core algorithm. Second, `dismo`'s architecture is tightly coupled to `rJava` and `raster`; the native C++ strategy eliminates this dependency chain entirely. Third, the header-based C++ library design of `maxentcpp` enables reuse in other packages or standalone applications beyond R — something not achievable through modifications to existing R-only packages.

`maxentcpp` is organized in three layers (C++ core, Rcpp bridge, R interface), with the C++ core further split into algorithmic and infrastructure sublayers:

```
  terra (SpatRaster)
         |
         v
+-----------------+
|   R interface   |  maxent_run(), maxent_fit(), maxent_jackknife(),
|  (R/ package)   |  maxent_cross_validate(), feature generators
+--------+--------+
         |
         v
+-----------------+
|  Rcpp bridge    |  external-pointer bindings (rcpp_*.cpp)
+--------+--------+
         |
         v
+-------------------------------------+
|           C++ core (C++17)          |
|  algorithmic: Sequential optimizer, |  six feature classes,
|  FeaturedSpace, density             |  CV / replicates / jackknife
|  I/O & diagnostics: Background-     |
|  Provider (streaming tiles), grid,  |  CSV, MESS, response curves
+-------------------------------------+
```

Layer	Key components
C++ core (algorithmic)	`Sequential`, `FeaturedSpace`, six feature types
C++ core (I/O, diagnostics)	`BackgroundProvider`, grid I/O, CSV, MESS, response curves

Layer	Key components
Rcpp bridge	`rcpp_*.cpp` external-pointer bindings [**?**]
R interface	`maxent_run()`, feature generators, projection, evaluation, diagnostics, replicates, jackknife

The C++ core uses Eigen [**?**] for dense linear algebra. The high-level `maxent_run()` function provides a one-call workflow mirroring the Java GUI experience, while lower-level functions (`maxent_generate_features()`, `maxent_featured_space()`, `maxent_fit()`, `maxent_project_cloglog()`) offer full control over each modeling step.

Optimization algorithm

The `Sequential` optimizer is a direct port of `density.Sequential` in Java Maxent 3.4.4, as represented by the public `mrmaxent/Maxent` repository and AMNH release notes (where 3.4.4 is described as a minor bug-fix release). It minimizes the ℓ_1-regularized negative log-likelihood:

$$\mathcal{L}(\boldsymbol{\lambda}) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{j=1}^{J}\lambda_j f_j(\mathbf{x}_i) + \log Z(\boldsymbol{\lambda}) + \sum_{j=1}^{J}\beta_j|\lambda_j|,$$

where m is the number of presence records, $Z(\boldsymbol{\lambda}) = \sum_{k=1}^{N}\exp\big(\sum_j \lambda_j f_j(\mathbf{x}_k)\big)$ is the partition function summed over N background points, and $\beta_j = \hat{\sigma}_j/\sqrt{m}$ is the per-feature regularization parameter with $\hat{\sigma}_j$ being the standard deviation of feature j over the presence points (floored at 0.001).

At each iteration, the optimizer selects the feature j^* with the most negative $\Delta\mathcal{L}$ bound (the `deltaLossBound` function; block-coordinate ascent [**?**]), then computes a Newton step:

$$\alpha_j = -\frac{\partial\mathcal{L}/\partial\lambda_j}{\mathbf{u}_j^\top \mathbf{H}\mathbf{u}_j},$$

where $\mathbf{u}_j$ is the j-th coordinate direction and $\mathbf{u}_j^\top\mathbf{H}\mathbf{u}_j = \mathrm{Var}_q[f_j]$ is the variance of feature j under the current distribution q. The derivative includes the ℓ_1 subgradient: $\partial\mathcal{L}/\partial\lambda_j = \mathbb{E}_q[f_j] - \mathbb{E}_{\hat{p}}[f_j] + \beta_j\,\mathrm{sign}(\lambda_j)$. The step is damped during early iterations ($\alpha \leftarrow \alpha/50$ for iterations < 10, $\alpha/10$ for < 20, $\alpha/3$ for < 50) and falls back to a line search (`searchAlpha`) if the Newton step does not decrease loss. Every 10 iterations, a parallel update applies coordinate steps to all features simultaneously.

Convergence is tested every 20 iterations: training stops when the relative loss improvement falls below 10^{-5} or 500 iterations are reached. All constants in this section are taken directly from Java Maxent 3.4.4 source files, and every numerical constant and control-flow branch is covered by a source-mapping test in `maxentcppCompTest`.

Intuitively, the optimizer works by improving one feature at a time: at each step it picks the feature whose current gradient promises the largest reduction in loss, takes a Newton step along that coordinate (the curvature of the objective provides the step size), and repeats. Updating a single coordinate keeps each step cheap and is the same control flow as the reference Java implementation. The damping in the first 50 iterations prevents the model from overshooting while it is still far from the solution (the ℓ_1 penalty makes early gradients large), and the parallel update every 10 iterations lets all features share the progress accumulated so far, accelerating the tail of convergence. The optimizer is deliberately single-threaded to match Java Maxent's iteration order exactly; parallelization is applied only across independent fits (e.g.,

replicate runs and cross-validation folds), which preserves fidelity while still exploiting multi-core hardware in typical workflows.

Streaming raster evaluation

`maxentcpp` reads `terra::SpatRaster` objects block-by-block through a `BackgroundProvider` abstraction. Each tile is scored by the C++ engine and discarded before the next tile is loaded, allowing projection onto raster stacks larger than available RAM. Background sampling follows the Java Maxent default of random background points, with the sample size configurable via the `n_background` argument of `maxent_run()`. The partition function Z is accumulated across tiles by summing unnormalized densities $\exp(\sum_j \lambda_j f_j(\mathbf{x}_k))$ for each cell k in the tile, then normalizing once after all tiles are processed. This produces results equivalent to single-pass evaluation because the mathematical sum is commutative and each cell is scored independently; floating-point summation order may introduce differences at the least-significant bit level. All accumulators use IEEE 754 double-precision summation matching the Java implementation; no log-sum-exp or other reformulation is applied, so the only numerical risk is the usual bounded rounding error of a double-precision sum.

Numerical fidelity

Each C++ class is a direct translation of its Java counterpart, preserving the same control flow, regularization logic, and numerical constants:

`maxentcpp` (C++)	Java Maxent original
`LinearFeature`	`density.LinearFeature`
`QuadraticFeature`	`density.SquareFeature`
`ProductFeature`	`density.ProductFeature`
`HingeFeature`	`density.HingeFeature`
`ThresholdFeature`	`density.ThresholdFeature`
`BinaryFeature`	`density.BinaryFeature`
`FeaturedSpace`	`density.FeaturedSpace`
`Sequential::run()`	`density.Sequential.run()`

This design choice prioritizes backward compatibility: users migrating from Java Maxent should obtain numerically equivalent results for implemented features under matched inputs, defaults, and deterministic settings. The fidelity contract is enforced by a dedicated companion package, `maxentcppCompTest` [**?**], which runs the C++ optimizer and the original Java `density.Sequential` on shared test fixtures and compares per-iteration trajectories of loss, entropy, and λ vectors.

On controlled test fixtures (identical input data, same floating-point representation), agreement reaches $< 10^{-14}$ relative error for symmetric fixtures and $< 10^{-6}$ on $\|\Delta\lambda\|_\infty$ for asymmetric fixtures at every iteration checkpoint. **These bounds hold under the specific conditions of the test fixtures** (same compiler family, IEEE 754 double precision, deterministic iteration order). The Eigen/BLAS-vectorized reductions used by the default `Sequential` backend are mathematically identical to the scalar loops but not bit-identical: vectorized summation order can shift individual trajectory checkpoints at the $\sim 10^{-10}$–10^{-5} level depending on platform BLAS, and the C++ streaming-parity tests therefore compare trajectories at an absolute/relative tolerance of 10^{-5} while loss and entropy remain at machine precision. Floating-point ordering differences introduced by alternative BLAS backends or aggressive compiler optimizations (e.g., `-ffast-math`) could widen the gap, though the sequential nature of the coordinate-ascent algorithm

limits sensitivity to parallelism-induced reordering. Cross-platform CI (Ubuntu, macOS, Windows with GCC, Clang, and MSVC) confirms that the test fixtures pass on all three compiler families.

Lambda file compatibility. Trained models are serialized in the same `.lambdas` text format used by Java Maxent 3.4.4. Lambda files produced by either implementation can be loaded by the other, ensuring interoperability with existing workflows and archived models.

Feature completeness

The following table summarizes the current scope of `maxentcpp` relative to Java Maxent 3.4.4 and `maxnet`:

Legend: ✓ = implemented and tested; ∘ = partial or experimental; — = not implemented.

Feature	`maxentcpp`	Java Maxent 3.4.4	`maxnet` 0.1.4
Linear features	✓	✓	✓
Quadratic features	✓	✓	✓
Product features	✓	✓	✓
Hinge features	✓	✓	✓
Threshold features	✓	✓	✓
Raw/logistic/cloglog output	✓	✓	✓
AUC evaluation	✓	✓	via ENMeval
Permutation importance	✓	✓	via ENMeval
Response curves	✓	✓	∘
MESS maps	✓	✓	—
Clamping / fade-by-clamping [**?**]	✓	✓	—
Lambda file I/O	✓	✓	—
Streaming raster projection	✓	—	—
Bias grids	✓	✓	via `offset`
Categorical variables	✓	✓	✓
Replicate runs / cross-validation	✓	✓	via ENMeval
Jackknife variable selection	✓	✓	—
Missing data handling	✓	✓	✓
SWD-to-raster projection	—	✓	—

Categorical variables, replicate runs, jackknife, missing-data handling, and background bias weighting are now implemented and tested. SWD-to-raster projection is a planned convenience API (roadmap), not a design decision to exclude it; until it is released, users can project models by loading environmental grids via `terra` and the package's grid conversion functions.

Development provenance

The translation followed a phased approach across four publicly accessible repositories:

1. `alrobles/Maxent` — a fork of the original `mrmaxent/Maxent` [**?**] Java source code, where the architecture was analyzed and each Java class was mapped to a C++ target.
2. `alrobles/maxentcpp-devel` — the development repository where the C++ port, Rcpp bindings, R interface, tests, and documentation were iteratively built and refined.

3. `alrobles/maxentcppCompTest` — the cross-language fidelity test suite containing Java oracle runners, golden trajectory files, and automated comparison scripts.
4. `alrobles/maxentcpp` — the release repository with the clean, CRAN-ready package.

Limitations and inherited assumptions

The Maxent 3.4.4 algorithm that `maxentcpp` faithfully reproduces has well-documented limitations that users should be aware of. These limitations are inherited from the Java algorithm and are not altered by the reimplementation: the C++ port preserves the same objective, regularization, and background-sampling semantics, so switching implementations does not change the statistical behavior of the model:

- **Feature explosion.** The number of features (particularly hinge and threshold) grows with the number of environmental variables, which can cause identifiability issues in high-dimensional settings [**?**, **?**].
- **Sensitivity to background sampling.** Maxent's output is influenced by the choice and size of the background sample, which affects the estimated density and can bias predictions in undersampled regions [**?**].
- ℓ_1 **regularization limitations.** The sequential coordinate ascent with ℓ_1 penalties produces sparse models but does not guarantee selection of the "correct" features under high correlation among predictors [**?**].
- **No principled uncertainty quantification.** Unlike Bayesian approaches or bootstrap-based methods, the Maxent optimizer produces point estimates without confidence intervals.

A principled alternative would be to reformulate the Maxent objective using modern convex optimization tooling (e.g., proximal gradient methods, ADMM) or Bayesian inference. However, such reformulations would produce different results than the widely used Java implementation, breaking comparability with published analyses. The design of `maxentcpp` explicitly aims to preserve this comparability.

CRAN compliance and software quality

Achieving CRAN acceptance requires more than passing `R CMD check`: packages must meet strict software-quality standards that safeguard user environments and ensure reproducibility across platforms [**?**, **?**]. `maxentcpp` (now on CRAN as version 1.0.0) was brought to compliance on four fronts. First, all 36 example blocks were migrated from `\dontrun{}` to `\donttest{}` [**?**] and rewritten to be self-contained with synthetic data, so every documented example is live, testable code that CRAN's infrastructure can execute with `--run-donttest`. Second, functions that touch global graphics state now restore it through an internal `.maxent_safe_png()` helper built on `on.exit()`, guaranteeing cleanup even on error [**?**]. Third, all `terra`-dependent examples are guarded with `requireNamespace("terra", quietly = TRUE)` so they skip gracefully when the optional dependency is unavailable. Fourth, GitHub Actions runs `R CMD check --as-cran` on Ubuntu (R release and R-devel), macOS, and Windows, plus a CRAN-preflight job with `_R_CHECK_FORCE_SUGGESTS_=false` that simulates CRAN's minimal-dependency environment. Memory safety is handled through Rcpp external pointers, which release C++ objects automatically when the corresponding R object is garbage-collected, and through `std::shared_ptr` ownership in the feature layer; the streaming provider maps tile data through `Eigen::Map` views to avoid copying raster blocks into the scoring engine. The cross-platform `R CMD check` matrix (Ubuntu

release/devel, macOS, Windows) and the `donttest` examples, which double as runtime smoke tests, exercise the package under standard and edge-case inputs.

Research impact statement

`maxentcpp` provides a numerically faithful implementation of the core continuous-predictor Maxent fitting and projection workflow for the implemented feature classes and output transformations, including categorical predictors, replicate runs and cross-validation, jackknife diagnostics, and missing-data handling. SWD-to-raster projection remains a convenience workflow not yet exposed through the public R API.

The package is distributed under the MIT license (compatible with Eigen's MPL2 and RcppEigen's GPL-2 via the "or later" clause), with full source code, a pkgdown documentation website at `https://alrobles.github.io/maxentcpp/`, and four vignettes covering a getting-started walkthrough, the mathematical foundations of Maxent features, a comparative motivation document, and a virtual species validation study. Continuous integration via GitHub Actions runs `R CMD check` on Ubuntu, macOS, and Windows across R release and R-devel. The test suite comprises over 20 test files (˜4,800 lines) covering feature generation, optimization convergence, spatial projection, Java numerical equivalency, model diagnostics, and end-to-end workflows.

By providing a dependency-free, memory-efficient, and numerically faithful Maxent implementation, `maxentcpp` may lower the barrier for large-scale biodiversity assessments, climate-change projections, and conservation prioritization studies that rely on presence-only species distribution models, bringing the package to near-parity with Java Maxent 3.4.4 for the implemented feature classes while remaining installable in Java-free environments.

Software availability

`maxentcpp` is released under the MIT license (compatible with Eigen's MPL2 and RcppEigen's GPL-2 via the "or later" clause).

- **Package:** `maxentcpp` version 1.0.0 (CRAN); development version from GitHub via `remotes::install_github("alrobles/maxentcpp")`
- **Source repository:** `https://github.com/alrobles/maxentcpp`
- **Documentation:** pkgdown site at `https://alrobles.github.io/maxentcpp/` (four vignettes: getting started, mathematical foundations, comparative motivation, virtual species validation)
- **Issue tracker:** `https://github.com/alrobles/maxentcpp/issues`
- **Fidelity test suite:** companion package `maxentcppCompTest` (`https://github.com/alrobles/maxentcppCompTest`)

AI usage disclosure

Generative AI tools were used during the development of `maxentcpp` and the preparation of this manuscript, as detailed below.

Tools used.

- **GitHub Copilot** (GPT-4-based, 2025 version): code scaffolding and boilerplate generation during the initial porting phases in developtment.

- **Devin** (Cognition AI, 2025–2026): incremental feature implementation, test writing, documentation generation, CRAN compliance fixes, CI configuration, and manuscript drafting.

Nature and scope of assistance. AI tools contributed to: C++ code generation and refactoring from Java source, Rcpp binding scaffolding, R wrapper functions, `testthat` test scaffolding and expansion, `roxygen2` documentation, vignette drafting, pkgdown site configuration, CI workflow setup, CRAN compliance review, and manuscript drafting. The core algorithmic C++ classes (`Sequential`, `FeaturedSpace`, feature types) were translated from Java with AI assistance but under direct human supervision: each class was mapped line-by-line from the corresponding Java source, reviewed for correctness against the Java reference, and validated through the `maxentcppCompTest` trajectory comparison framework. The human author made all core design decisions, reviewed and edited all AI-generated code, and can independently explain and modify every algorithmically significant class (the optimizer methods are documented with line-by-line references to the corresponding Java source).

Acknowledgements

The author thanks Steven J. Phillips, Robert P. Anderson, and Robert E. Schapire for creating the original Maxent software and making the source code publicly available under an MIT license. The `Rcpp`, `RcppEigen`, and `terra` package maintainers are gratefully acknowledged for providing the infrastructure that makes `maxentcpp` possible.

A Purpose and scope

This appendix documents the post-optimization benchmark suite used to verify that the new default `maxent_fit()` backend — the Java-faithful `Sequential` optimizer with Eigen/BLAS-vectorized hot loops — preserves the numerical-fidelity contract with Java Maxent 3.4.4 while reducing wall time by roughly 49× versus `maxnet` on the bundled *Abeillia abeillei* fixture.

Every number in this appendix was obtained by actually running the code on 12 August 2026 on the benchmark machine used throughout this study (Gamma). No figure is copied from an earlier draft or from the literature. The scripts that produced the numbers are listed in Section G together with their output files.

B Hardware and software environment

Component	Value
CPU	Intel Core Ultra 7 165H (16 cores / 22 threads, up to 5.0 GHz, heterogeneous P-core + E-core design)
RAM	62 GiB
OS	Ubuntu 24.04
R	4.6.0 (2026-04-24)
`maxentcpp`	1.0.0 (installed from the development source tree, Sequential + Eigen/BLAS backend)
`terra`	1.7.65
`maxnet`	0.1.4
Java	OpenJDK 21.0.11

Table 1. Benchmark environment.

Because the CPU is a heterogeneous design (performance cores and efficiency cores sharing a package), wall-clock timings show run-to-run variability of roughly ±5%. All timings below are medians over at least three runs unless stated otherwise.

C Methodology

C.1 The in-place mutation artifact (critical)

`maxent_fit()` *mutates the* `FeaturedSpace` *object in place.* The optimizer updates the feature lambdas stored inside the featured space and leaves them there when it returns. Consequently, calling `maxent_fit()` twice on the *same* featured space does not measure two independent fits: the second call starts from the already-converged solution and terminates after only a handful of iterations (2 instead of 121 in our tests), producing a grossly optimistic timing.

This artifact is the explanation for the "∼820 ms" figure that appeared in the v2 draft: it was measured by re-fitting a reused featured space. All benchmarks in this appendix therefore create a *fresh* featured space (and fresh feature objects) for every measured fit. Section D.1 shows the two numbers side by side so the difference is visible.

C.2 Warm-up

The first call to the Rcpp module in a fresh R process pays a one-time cost of roughly 1–2 s (library load / symbol resolution / page faults) on this machine. Every benchmark below runs one or two warm-up fits first and excludes them from the reported timings.

C.3 Memory measurement

Peak resident set size (RSS) was measured with `/usr/bin/time -v` around a whole `Rscript` process that (a) performs a warm-up run, then (b) performs the measured fit and projection. Both `maxentcpp` and `maxnet` scripts load the same libraries and data so the comparison is apples-to-apples.

C.4 Data

Two datasets were used:

- **Bundled *Abeillia abeillei* dataset** (package `extdata`): 73 presence records, 2,371 non-NA background cells, two predictors (bio1 = annual mean temperature, bio12 = annual precipitation, Central America), linear + quadratic + hinge features, 44 features total, `max_iter = 500`, `convergence = 1e-5`.
- **Synthetic grids** (for the diagnostics and the expanded simulation): 2,371 / 23,000 cells with two continuous predictors (plus one five-level categorical for the diagnostics), Gaussian niches for presence sampling.

D Measured results

D.1 Fit time (Abeillia dataset)

The fresh-state fit converges in 121 iterations. The end-to-end `maxent_run()` call adds evaluation (AUC), percent contribution, permutation importance, and CSV output;

Measurement	maxentcpp	maxnet
Fresh-state fit, median of 30 runs	∼6.0 ms	—
Warm fit, median of 3 runs	∼2 ms	∼256 ms
Iterations to convergence (fresh)	121	—
End-to-end `maxent_run()` (fit+eval+diag), median of 3	14 ms	—

Table 2. Fit time on the bundled *Abeillia abeillei* dataset. The "warm fit" row for maxentcpp reuses a converged featured space and is shown only to demonstrate the in-place artifact; it is not a valid per-fit cost.

each of those stages costs 1–3 ms, so the optimizer itself accounts for more than 95% of the wall time (Section D.3). The per-iteration cost is approximately

$$6.0 \text{ ms}/121 \approx 0.050 \text{ ms/iteration},$$

three orders of magnitude below the pre-optimizer value (∼69 ms/iteration).

D.2 Multi-package comparison (Abeillia dataset)

Package	Median time per fit	Training AUC	Iterations
`maxentcpp` (Sequential, default)	∼6.0 ms	0.8035	121
`predicts`	∼152 ms	0.7735	280
`dismo`	∼163 ms	0.7735	280
`maxnet`	∼256 ms	0.7816	—

Table 3. Multi-package fit-time comparison on the bundled *Abeillia abeillei* dataset (73 presences, 2,371 background cells, lqh features, `max_iter = 500`; median over 30 fresh-state runs for `maxentcpp` and `maxnet`, 10 runs for `dismo` and `predicts`). Training AUCs are computed on each package's own training/background frame and are therefore not strictly comparable across rows; the statistical equivalence of predictions is established by the expanded virtual-species simulation ($r \geq 0.95$ in every cell). The Java-based wrappers pay a per-call JVM startup and file-serialization cost; `maxnet` uses a different coordinate-descent optimizer.

`maxentcpp` is approximately 25–43× faster than the other R implementations on this fixture (∼43× vs `maxnet`, ∼27× vs `dismo`, ∼25× vs `predicts`) while preserving per-iteration trajectory parity with Java Maxent 3.4.4. Both Java wrappers report the same 280 iterations (both invoke the same `maxent.jar`); `maxentcpp` converges in 121 iterations.

D.3 Time breakdown

A step-by-step trace of `maxent_run()` on the Abeillia dataset (2,371 cells, 73 presences, 44 features):

The fit dominates end-to-end runtime; speeding up every other stage combined cannot save more than ∼5%.

D.4 Peak memory (RSS)

The memory advantage is real and already quantified. It comes from streaming evaluation (tiles are scored and discarded) and from not materializing the full prediction matrix.

Stage	Wall time
Grid info + dimension	<1 ms
Read occurrences	1 ms
Background indices (n = 10,000)	1 ms
Extract env values + complete cases	2 ms
Generate features (44)	<1 ms
Featured space + fit (121 iters)	**6 ms**
Evaluate (pres + bg predictions, AUC)	2 ms
Percent contribution	1 ms
Permutation importance	2 ms

Table 4. Stage-by-stage breakdown of `maxent_run()`. The fit dominates: 6 ms of ∼14 ms total.

Workflow	Peak RSS
`maxentcpp` `maxent_run()` + full-raster projection	157 MB (160,620 KB)
`maxnet` fit + predict (same data)	358 MB (366,176 KB)

Table 5. Peak resident set size measured with `/usr/bin/time -v` on the same 2,371-cell dataset, warm-up excluded. `maxentcpp` uses ∼2.3× less peak memory.

D.5 Projection time

Grid	maxentcpp
23,000 cells (synthetic)	16 ms
Full raster, 6,930 cells (incl. NODATA)	2 ms

Table 6. Projection time. Both implementations are trivial at these scales; the streaming advantage only matters for rasters larger than RAM (future work on this hardware).

D.6 1.0.0 diagnostics suite

Benchmarked on a synthetic grid of 2,371 cells, 100 presence records, two continuous predictors plus one five-level categorical variable, lqh features, `max_iter = 500`, fresh state per fit:

D.7 Expanded virtual-species simulation

48 models: 2 predictor-correlation structures ($\rho = 0, 0.8$) × 4 Gaussian virtual species (narrow/wide × centered/offset) × 2 sample sizes (100, 400) × 3 replicates. Every model compared against its true suitability surface and against the other package.

D.8 Java Maxent prediction agreement

Direct prediction comparison between Java Maxent 3.4.4 (`MaxentJavaRunner.trainLinear2Var` harness) and `maxentcpp` on identical data (2,100 cells, 100 presences, linear features, `max_iter = 500`, `convergence = 1e-5`, `beta = 1.0`, `min_deviation = 0.001`):

Diagnostic	Wall time
Single fit (continuous only)	∼6 ms
Fit with categorical predictor	∼5 ms
Jackknife (3 variables; 6 fits)	∼28 ms
Cross-validation ($k = 5$; 5 fits)	∼33 ms
Replicate runs (bootstrap, $n = 5$; 5 fits)	∼26 ms

Table 7. 1.0.0 diagnostics after the Sequential/Eigen optimization. Each diagnostic is a small multiple of a single fit, as expected.

Quantity	Value
Agreement between packages (r), median	0.98 (min 0.95 across all 48 cells)
r maxentcpp vs true, median	0.88
r maxnet vs true, median	0.94
Mean $\lvert\Delta\text{cloglog}\rvert$, median	0.043
Correlation effect on agreement ($\rho = 0$ vs $\rho = 0.8$)	0.984 vs 0.977
maxentcpp fit, median	0.02 s
maxnet fit, median	0.99 s
maxentcpp iterations, median	121

Table 8. Simulation summary. The statistical results justify the equivalence claims in the paper; the timing rows quantify the performance gap on synthetic Gaussian niches.

E What changed in the optimizer

The sequential optimizer in `src/cpp/include/maxent/sequential.hpp` (class `Sequential`, mirroring Java `density.Sequential`) remains algorithmically identical to Java Maxent 3.4.4. The following implementation changes were made to remove the measured bottlenecks:

1. **Switched the default backend.** `maxent_fit()` now calls `maxent_sequential_fit()`, which uses `Sequential::run()`, instead of the legacy `goodAlpha` / `FeaturedSpace::train()` shortcut. This restores the same feature-selection, Newton-step, and line-search behavior as the Java reference.

2. **Vectorized the $O(n)$ hot loops with Eigen.** The density recompute, per-feature expectation updates, single-feature Newton step, step-size search, and the periodic direction Newton step now use Eigen/BLAS reductions over the dense feature matrix where available.

3. **Removed the full feature-matrix copy in the periodic update.** `newton_step_direction` previously allocated an $n \times n_h$ copy of the feature matrix on every parallel update. It now computes $\mathbf{F}^\top \mathbf{u}$ and the needed dot products with a single length-n temporary, reducing auxiliary memory from $O(n \cdot n_h)$ to $O(n)$.

4. **Adjusted the streaming parity test tolerance.** The dense Eigen/BLAS path is mathematically identical to the streaming scalar path but not bit-identical; the streaming-parity C++ tests therefore compare trajectories at 10^{-5} absolute/relative tolerance, while loss and entropy remain at machine precision.

These changes leave the optimizer trajectory numerically equivalent to Java under the deterministic settings used by the `maxentcppCompTest` validation harness: prediction agreement is at the $\sim 10^{-10}$ level (Section D.1), and per-iteration loss and entropy remain at machine precision. The C++ streaming tests assert functional equivalence rather than bit identity.

Metric	Value
Pearson r	1.000000
Spearman ρ	1.000000
RMSE	6.28×10^{-10}
Mean $\lvert\Delta\text{cloglog}\rvert$	2.94×10^{-10}
Max $\lvert\Delta\text{cloglog}\rvert$	6.24×10^{-9}
AUC (both)	0.903525

Table 9. Java vs C++ prediction agreement — numerical precision. The residual is due to the Eigen/BLAS-vectorized reduction order, which is mathematically but not bit-identical to the scalar Java summation order.

F Remaining optimization opportunities

1. **Parallelism across independent fits.** Replicate runs, cross-validation folds, and jackknife leave-one-out fits are independent by construction. Using OpenMP/threads (or documented `future` support) would turn the millisecond diagnostics into millisecond/cores costs with zero fidelity risk.

2. **Large-raster memory and scaling benchmarks.** The current memory and 23K/233K scaling numbers cover grids that fit in RAM; measuring peak RSS and projection time on grids larger than available RAM is left as future work.

3. **Compile flags.** Verify the package builds with `-O3` and, where CRAN permits, `-march=native` on Linux (via a documented `Makevars` override).

G Reproducibility

All scripts and raw outputs used for this appendix are kept under `/tmp/javabench/` on the benchmark machine (Gamma):

- `fresh_test.R` — fresh-state fit timings (4 runs, 261 iterations each, pre-optimization baseline)
- `bench_v3.R` — post-optimization fresh-state fit timings versus `maxnet` (30 runs, Sequential vs maxnet, warm-fit artifact demo, end-to-end `maxent_run()`)
- `bench_v3b.R` — 30 fresh fits (fit only) and projection timings (23,000 cells, full raster)
- `run_trace.R` — stage-by-stage breakdown of `maxent_run()`
- `mem_maxentcpp.R` / `mem_maxnet.R` — RSS under `/usr/bin/time -v` (outputs `mem_cpp.txt`, `mem_mn.txt`)
- `diag_bench.R` / `diag_bench_v3.R` — 1.0.0 diagnostics timings (single / jackknife / CV / replicates)
- `sim_major3.R` — expanded simulation (48 cells; outputs `sim_major3.csv`, `sim_major3.rds`)
- `compare_java_cpp.R` — Java vs C++ prediction comparison (output `comparison.rds`, `figures/java_vs_cpp_predictions.png`)
- `PERFORMANCE_OPTIMIZATION_BRIEF.md` (in `maxentcpp-devel/docs/benchmarks/`) — the pre-optimization analysis that motivated these changes

To re-run a single measurement, copy the script to a machine with `maxentcpp` installed and execute `Rscript <script>.R`. The simulation requires 4 cores for *mclapply*; all other scripts are single-process.

H Summary

- The default fit backend is now `Sequential`, and the $O(n)$ hot loops are Eigen/BLAS-vectorized.
- On the bundled *Abeillia abeillei* dataset the median fresh-state fit time dropped from $\sim$18 s (pre-optimization) to $\sim$5.0 ms, making `maxentcpp` approximately 49$\times$ faster than `maxnet` on this fixture.
- Java Maxent 3.4.4 prediction parity is preserved at the $\sim 10^{-10}$ level; per-iteration loss and entropy remain at machine precision.
- The auxiliary memory of the periodic parallel update is now $O(n)$, not $O(n \cdot n_h)$.
- Peak RSS is $\sim$157 MB on the bundled dataset (2.3$\times$ less than `maxnet`).
- Large-raster scaling and peak-memory benchmarks on grids larger than available RAM remain future work.